

DOCKER

COMPLETE GUIDE TO DOCKER FOR BEGINNERS
AND INTERMEDIATES

Questions and Answers



DAY 4

MASTER DEVOPS WITH VINAY

31: How do you create a Docker volume and mount it into a container?

Answer:

Steps:

1. Create a volume:
`docker volume create my-volume`
2. Run a container with the volume mounted:
`docker run -v my-volume:/app --name my-container alpine`
3. Inspect the volume:
`docker volume inspect my-volume`

Explanation:

- The `-v` flag attaches the named volume `my-volume` to the `/app` directory inside the container.
- Any data stored in `/app` will be retained even if the container is removed, ensuring data persistence across container lifecycles.

32: How are services specified in a docker-compose.yml file?

Answer:

Services are defined under the `services` section in a `docker-compose.yml` file.

Each service corresponds to a container.

Example:

version: '3.8'

services:

web:

image: nginx

ports:

- "8080:80"

database:

image: postgres

environment:

POSTGRES_USER: admin

POSTGRES_PASSWORD: password

Explanation:

- `web`: Defines an NGINX container accessible on host port 8080, mapped to container port 80.
- `database`: Sets up a PostgreSQL container with environment variables for username and password.

33: Can you explain the difference between docker-compose up and start commands

Answer:

Command	Description
<code>docker-compose up</code>	Creates and starts containers, networks, and volumes as defined in the <code>docker-compose.yml</code> file.
<code>docker-compose start</code>	Starts existing containers without creating new ones.

Explanation:

- Use `docker-compose up` when launching the application for the first time or after making changes to the configuration.
- Use `docker-compose start` to restart previously created containers without rebuilding or recreating them.

34: How do you scale services using Docker Compose?

Answer: Docker Compose supports scaling by allowing you to run multiple instances (replicas) of a service.

Command: `docker-compose up --scale <service-name>=<number-of-replicas>`

Example:

`docker-compose up --scale web=3`

This command scales the `web` service to 3 replicas.

Note:

- Scaling only works for services without the `depends_on` directive in the `docker-compose.yml` file.

35: How do you override default configurations in Docker Compose?

Answer:

Docker Compose allows you to override default settings by using an additional file, commonly named `docker-compose.override.yml`.

Steps:

1. Create an override file named `docker-compose.override.yml`.
2. Run `docker-compose` up normally.
Docker Compose will automatically merge the configurations from both files.

Example:

`docker-compose.yml`:

```
services:
  web:
    image: nginx
    ports:
      - "8080:80"
```

`docker-compose.override.yml`:

```
services:
  web:
    environment:
      - DEBUG=true
```

Explanation:

- The override file adds or replaces configuration for the same service (web in this case).
- This is useful for environment-specific customizations like enabling debug mode during development.

36: How does Docker Compose manage multi-container applications?

Answer:

Docker Compose manages multi-container applications using a `docker-compose.yml` file, which allows you to define and coordinate multiple services, networks, and volumes in a single configuration.

Key Features:

1. Service Definition: Define multiple interconnected services within one file.
2. Networking: Automatically sets up a shared network, enabling seamless communication between containers.
3. Data Sharing: Use named volumes to persist and share data across containers.
4. Scaling: Easily scale services horizontally using the `--scale` option.

37: What are Docker Secrets, and how do you use them?

Answer:

Docker Secrets are used to securely store and manage sensitive information, such as passwords, API keys, and TLS certificates, within a Docker Swarm cluster.

Key Features:

- Secure Storage
- Secrets are encrypted and stored securely, ensuring they're only available to services that explicitly need them.

2. Access Control

- Only containers running in Swarm mode and granted permission can access the secrets.

✔ Steps to Use Docker Secrets:

1. Create a secret: `echo "my-secret-password" | docker secret create my_secret -`
2. Use the secret in a service: `docker service create --name my-service --secret my_secret nginx`
3. Access the secret inside the container: Secrets are automatically mounted as files under:
`/run/secrets/my_secret`

38: What is the purpose of depends_on in a docker-compose.yml file?

Answer:

The `depends_on` keyword is used to control the startup order of services in Docker Compose. It ensures that a specified service starts after the services it depends on.

Example:

```
services:
  db:
    image: mysql

  app:
    image: my-app
    depends_on:
      - db
```

Explanation:

- In this example, the app service will start after the db service.

Important Note:

- `depends_on` only manages the order of startup, not the readiness of a service (e.g., it doesn't wait for the database to be fully initialized and accepting connections).

39: What is Docker Swarm, and how is it different from Kubernetes?

Answer:

Docker Swarm is Docker's native clustering and orchestration tool. It allows you to manage multiple Docker nodes as a single, unified cluster.

◆ Key Features of Docker Swarm:

1. Cluster Management
 - Automatically schedules and distributes containers (tasks) across multiple nodes.
2. Service Discovery
 - Includes built-in DNS-based service discovery.
3. Scaling
 - Easily scale services up or down with a single command.
4. Load Balancing
 - Distributes incoming traffic evenly across available service replicas.

◆ Docker Swarm vs. Kubernetes – >Key Differences:

Feature	Docker Swarm	Kubernetes
Complexity	Simple and quick to set up	More complex setup and configuration
Scalability	Suitable for small to medium setups	Designed for large-scale, high-availability
Ecosystem	Limited features and integrations	Rich ecosystem with advanced capabilities
Use Case	Small to medium workloads	Enterprise-grade, production-level systems

How do you deploy a Dockerized application in a Swarm cluster?

Answer:

To deploy a Dockerized application in a Docker Swarm cluster, follow these steps:

1. Initialize the Swarm:


```
docker swarm init
```
2. Create a Service:


```
docker service create --name my-service -p 8080:80 nginx
```
3. Scale the Service:


```
docker service scale my-service=3
```
4. Check Service Status:


```
docker service ls
```
5. Inspect Running Tasks:


```
docker service ps my-service
```

Explanation:

- This process sets up a highly available, load-balanced service (nginx in this case) with 3 replicas running across the Swarm nodes.

40: Explain multi-stage builds in Docker.

Answer:

Multi-stage builds allow you to use multiple FROM statements within a single Dockerfile to separate the build environment from the final runtime image. This technique helps create lean, production-ready images.

Benefits:

1. Smaller Image Size

Build tools and temporary dependencies are excluded from the final image.

1. Cleaner & Secure Production Image

Only the necessary binaries or artifacts are copied, reducing the attack surface.

Example: Multi-Stage Build in a Dockerfile

Stage 1: Build Stage

FROM golang:1.17 AS builder

WORKDIR /app

COPY . .

RUN go build -o myapp

Stage 2: Final Production Image

FROM alpine:latest

WORKDIR /app

COPY --from=builder /app/myapp .

CMD ["/myapp"]

Explanation:

- The first stage (golang:1.17) compiles the application.
- The second stage (alpine) includes only the compiled binary, making the image lightweight and production-ready.